

Differential equations

Equations of motion with constant acceleration are based on the differential equations: $v(t) = \frac{dx(t)}{dt}$, $a(t) = \frac{dv(t)}{dt}$. Integrating for $a(t) = \text{constant}$:

$$a(t) = \text{constant} \quad (1)$$

$$v(t) = \int a dt = \boxed{at + v_0} \quad (2)$$

$$x(t) = \int v dt = \boxed{\frac{1}{2}at^2 + v_0t + x_0} \quad (3)$$

For any interval $[t_1, t_2]$ and for $\Delta t = (t_2 - t_1) \dots$

$$x(t) \Big|_{t_1}^{t_2} = \int_{t_1}^{t_2} v dt = \left(\frac{1}{2}at_2^2 + v_0t_2 + x_0 \right) \quad (4)$$

$$- \left(\frac{1}{2}at_1^2 + v_0t_1 + x_0 \right) \quad (5)$$

$$= \frac{1}{2}a(t_2^2 - t_1^2) + v_0(t_2 - t_1) \quad (6)$$

$$= \frac{1}{2}a(t_2 - t_1)(t_2 + t_1) + v_0(t_2 - t_1) \quad (7)$$

$$= \boxed{\frac{1}{2}a(\Delta t)(t_2 + t_1) + v_0(\Delta t)} \quad (8)$$

Difference equations

To implement a difference equation version, calculate the initial versions of x_i , v_i , and t_i based on x_0 , v_0 , and t_0 . Then calculate the next versions x_{i+1} , v_{i+1} , and t_{i+1} based on the current versions x_i , v_i , and t_i and previous time t_{i-1} .

$$x_i = x_0 + v_0t_0 + \frac{1}{2}at_0^2 \quad (9)$$

$$v_i = v_0 + at_0 \quad (10)$$

$$t_i = t_0 \quad (11)$$

For $\Delta t = (t_i - t_{i-1}) \dots$

$$x_{i+1} = x_i + v_0(\Delta t) + \frac{1}{2}a(\Delta t)(t_i + t_{i-1}) \quad (12)$$

$$v_{i+1} = v_i + a(\Delta t) \quad (13)$$

$$t_{i+1} = t_i + (\Delta t) \quad (14)$$

Code

The following Python code (accel.py) demonstrates the instantaneous (continuous, differential) and recursive (discrete, difference) versions. They yield the same results.

```
1  #!/usr/bin/env python3
2  #
3  # accel.py
4  #
5
6  # initial values
7  x0, v0, t0, dt, a, tot = 3, 5, 7, 2, 10, 20
8
9  # instantaneous
10 for t in range(t0, tot + t0 + dt, dt):
11     # print('*', t, x0, v0 * t, a * t * t / 2)      # debug print
12     x = x0 + v0 * t + a * t * t / 2
13     v = v0 + a * t
14     print(f'{x:6.1f} {v:6.1f} {t:6.1f}')
15
16 print()
17
18 # recursive
19 xi, vi, ti = x0 + v0 * t0 + a * t0 * t0 / 2, v0 + a * t0, t0
20 print(f'{xi:6.1f} {vi:6.1f} {ti:6.1f}')
21 for t in range(t0 + dt, tot + t0 + dt, dt):
22     # print('*', ti, xi, v0 * dt, a * dt * (t + ti) / 2) # debug print
23     xi = xi + v0 * dt + a * dt * dt / 2                # *not* dt ** 2
24     xi = xi + v0 * dt + a * (t * t - ti * ti) / 2      # alternate version
25     vi = xi + v0 * dt + a * dt * (t + ti) / 2
26     vi = vi + a * dt
27     ti = ti + dt
28     print(f'{xi:6.1f} {vi:6.1f} {ti:6.1f}')
```

Which results (with these example initial values) in:

1	283.0	75.0	7.0
2	453.0	95.0	9.0
3	663.0	115.0	11.0
4	913.0	135.0	13.0
5	1203.0	155.0	15.0
6	1533.0	175.0	17.0
7	1903.0	195.0	19.0
8	2313.0	215.0	21.0
9	2763.0	235.0	23.0
10	3253.0	255.0	25.0
11	3783.0	275.0	27.0
12			
13	283.0	75.0	7.0
14	453.0	95.0	9.0
15	663.0	115.0	11.0
16	913.0	135.0	13.0
17	1203.0	155.0	15.0
18	1533.0	175.0	17.0
19	1903.0	195.0	19.0
20	2313.0	215.0	21.0
21	2763.0	235.0	23.0
22	3253.0	255.0	25.0
23	3783.0	275.0	27.0
24	>>>		

The code $xi = xi + v0 * dt + a * dt * dt / 2$ (that uses $(\Delta t)^2$ and is commented out) is not correct. The discrete calculation must include the last term of equation (12) to match the continuous calculation.

If $a(t) \neq \text{constant}$, then this analysis does not apply.